

Thực hành TOÁN HỌC TỔ HỢP

(Tài liệu này dành riêng cho mục đích học thuật)

Lab Đồ thị

Mục lục

- Thực hành
 - 1. Tổng quan về đồ thị
 - 1.1. Các thuộc tính của đồ thị
 - 1.2. Phân loại đồ thị
 - 1.3. Biểu diễn đồ thị
 - 1.4. Các đồ thị đặc biệt
 - 1.5. Vẽ đồ thị
 - 2. Các thuật toán trên đồ thị
 - 2.1. Thuật toán duyệt đồ thị theo chiều sâu Depth First Search (DFS)
 - 2.2. Thuật toán duyệt đồ thị theo chiều rộng Breadth First Search (BFS)
 - 2.3. Ứng dụng của thuật toán BFS và DFS
 - 2.3.1. Tìm đường đi
 - 2.3.2. Liên thông
 - 2.3.3. Cạnh cầu, đỉnh khớp
 - 2.3.4. Cây khung
 - 2.3.5. Phát hiện chu trình
 - 2.3.6. Kiểm tra đồ thị hai phía
 - 2.4. Thuật toán Dijkstra
 - 2.5. Thuật toán Bellman–Ford
 - 2.6. Thuật toán Floyd Warshall
 - 3. Cây và các thuật toán trên cây
 - 3.1. Tổng quan về cây
 - 3.1.1. Các thuộc tính của cây
 - 3.1.2. Phân loại cây
 - 3.1.3. Các cây đặc biệt
 - 3.1.4. Biểu diễn cây
 - 3.2. Các thuật toán trên cây

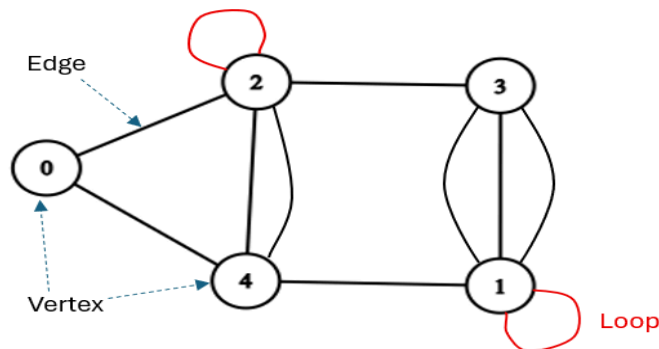
- 3.2.1. Thuật toán duyệt cây (Tree Traversal)
 - 3.2.2. Thuật toán Prim
 - 3.2.3. Thuật toán Kruskal
 - 3.2.4. Ứng dụng của thuật toán Prim và Kruskal
- 4. Luồng trong mạng
 - 4.1. Tổng quan
 - 4.2. Bài toán Max-Flow
 - 4.3. Thuật toán Ford–Fulkerson
 - 4.3.1. Xây dựng đồ thị dư
 - 4.3.2. Thuật toán Ford–Fulkerson
 - 4.4. Thuật toán Edmonds - Karp
 - 4.5. Thuật toán Dinic
- Đồ án
 - Yêu cầu
 - Quy định nộp bài
 - Quy định chấm bài
- Tài liệu tham khảo

Thực hành

1. Tổng quan về đồ thị

Một **đồ thị** (Graph) là một cặp $G = (V, E)$, với:

- V : tập các đỉnh (vertex/ vertices)
- E : tập các cạnh (directed edges/ undirected edges) nối giữa 2 đỉnh



Lưu ý:

- Null graph: là đồ thị không có cạnh
- Trivial graph: là đồ thị có duy nhất 1 đỉnh
- khuyên (loop): là cạnh có đỉnh đầu và đỉnh cuối trùng nhau
- Cạnh song song: là 2 cạnh có cùng đỉnh đầu, cùng đỉnh cuối

1.1. Các thuộc tính của đồ thị

Các thuộc tính về cạnh:

- Liên thuộc: cạnh $e = (u, v)$ nối 2 đỉnh u và v của đồ thị, khi đó ta nói e liên thuộc với u và v .
- Cạnh treo: là cạnh duy nhất tới đỉnh treo.

Các thuộc tính về đỉnh:

- Bậc của đỉnh: là số cạnh nối đỉnh đó với các đỉnh khác trong đồ thị, hay nói cách khác là số cạnh liên thuộc với đỉnh, ký hiệu: $\deg(v)$.
- Đỉnh treo: là đỉnh chỉ có duy nhất 1 cạnh liên thuộc với nó. Khi đó, $\deg(v) = 1$.
- Đỉnh cô lập: là đỉnh không có cạnh nào liên thuộc với nó. Khi đó, $\deg(v) = 0$.

1.2. Phân loại đồ thị

- Theo thiết kế cạnh: đồ thị có hướng và đồ thị vô hướng
- Theo đặc điểm cạnh: đơn đồ thị (không cạnh song song, không khuyên), đa đồ thị (có cạnh song song, không khuyên) và giả đồ thị (có cạnh song song, có khuyên)
- Theo số lượng cạnh: đồ thị đủ (mỗi đỉnh đều có cạnh nối với các đỉnh còn lại)
- Theo trọng số: đồ thị có trọng số (weights/ costs được gán trên cạnh) và đồ thị không có trọng số
- Theo tính liên thông: đồ thị liên thông (luôn có đường đi giữa 2 đỉnh) và đồ thị không liên thông
- Theo chu trình: đồ thị có chu trình và đồ thị không có chu trình

Đồ thị vô hướng:

- Bậc của đỉnh $A \in V$: $\deg(A) =$ tổng số cạnh (thuộc E) có chứa đỉnh A .
- Mọi đồ thị vô hướng đều có số đỉnh bậc lẻ là một số chẵn.

Đồ thị có hướng:

- Bậc vào đỉnh $A \in V$: $\deg^-(A) =$ số cạnh (thuộc E) có A là đỉnh cuối
- Bậc ra đỉnh $A \in V$: $\deg^+(A) =$ số cạnh (thuộc E) có A là đỉnh đầu

Bổ đề bắt tay (Hand shaking lemma): Cho đơn đồ thị $G = (V, E)$

- Nếu G vô hướng thì: $\sum_{A \in V} \deg(A) = 2|E|$
- Nếu G có hướng thì: $\sum_{A \in V} \deg^-(A) = \sum_{A \in V} \deg^+(A) = |E|$

1.3. Biểu diễn đồ thị

Biểu diễn bằng ma trận kề (ma trận đỉnh kề - Adjacency Matrix):

Cho đơn đồ thị $G = (V, E)$ vô hướng, $|V| = n$, $V = \{A_1, A_2, \dots, A_n\}$.

Ma trận kề ứng với G là $M_{n \times n}$ thỏa mãn:

$$M[i][j] = \begin{cases} 1, & \text{nếu } (i, j) \in E, \\ 0, & \text{nếu } (i, j) \notin E \end{cases}, \forall i, j$$

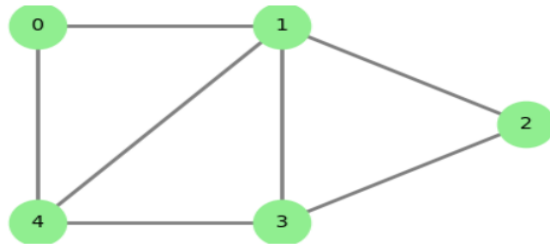
hoặc:

$$M[i][j] = \begin{cases} w, & \text{với } w \text{ là trọng số của cạnh } (i,j), \\ \infty, & \text{nếu không có cạnh } (i,j) \end{cases}, \forall i, j$$

Tính chất:

- M đối xứng
- $\sum_{j=1}^n M[i][j] = \sum_{j=1}^n M[j][i] = \deg(A_i), \forall i = 1, \dots, n$

Trong python, ma trận kề có thể được biểu diễn bằng kiểu dữ liệu `list`



```
In [1]: graph = [[0, 1, 0, 0, 1],
                 [1, 0, 1, 1, 1],
                 [0, 1, 0, 1, 0],
                 [0, 1, 1, 0, 1],
                 [1, 1, 0, 1, 0]]
graph
```

```
Out[1]: [[0, 1, 0, 0, 1],
          [1, 0, 1, 1, 1],
          [0, 1, 0, 1, 0],
          [0, 1, 1, 0, 1],
          [1, 1, 0, 1, 0]]
```

Hoặc sử dụng lớp `array` trong thư viện `numpy`

```
In [2]: import numpy as np
graph = np.array([[0, 1, 0, 0, 1],
                  [1, 0, 1, 1, 1],
                  [0, 1, 0, 1, 0],
                  [0, 1, 1, 0, 1],
                  [1, 1, 0, 1, 0]])
graph
```

```
Out[2]: array([[0, 1, 0, 0, 1],
               [1, 0, 1, 1, 1],
               [0, 1, 0, 1, 0],
               [0, 1, 1, 0, 1],
               [1, 1, 0, 1, 0]])
```

Biểu diễn bằng danh sách cạnh (Edge List):

Trong python, danh sách cạnh có thể được biểu diễn bằng kiểu dữ liệu `list`

```
In [3]: edges = [(0,1), (0,4),
                 (1,2), (1,3), (1,4),
                 (2,3),
                 (3,4)]
edges
```

```
Out[3]: [(0, 1), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (3, 4)]
```

Biểu diễn bằng danh sách kề (danh sách đỉnh kề - Adjacency List):

Trong python, danh sách đỉnh kề có thể được biểu diễn bằng kiểu dữ liệu `dictionary`

```
In [4]: graph = {0: [1,4],
                1: [0,2,3,4],
                2: [1,3],
                3: [1,2,4],
                4: [0,1,3]}
graph
```

```
Out[4]: {0: [1, 4], 1: [0, 2, 3, 4], 2: [1, 3], 3: [1, 2, 4], 4: [0, 1, 3]}
```

Biểu diễn bằng ma trận liên thuộc/ ma trận liên kết (Incidence Matrix):

Cho đồ thị $G = (V, E)$ vô hướng, không có khuyên,
 $|V| = n, V = \{A_1, A_2, \dots, A_n\}, |E| = m.$

Ma trận liên thuộc ứng với G là $Inc(G)_{n \times m}$ thỏa mãn:

- Với G vô hướng:

$$Inc(G)[i][j] = \begin{cases} 1, & \text{nếu đỉnh } v_i \text{ kề với cạnh } e_j, \\ 0, & \text{nếu ngược lại} \end{cases}, \forall i, j$$

- Với G có hướng:

$$Inc(G)[i][j] = \begin{cases} 1, & \text{nếu cạnh } u_j \text{ hướng ra khỏi đỉnh } x_i, \\ -1, & \text{nếu cạnh } u_j \text{ hướng vào đỉnh } x_i, \\ 0, & \text{nếu cạnh } u_j \text{ không kề đỉnh } x_i \end{cases}, \forall i, j$$

Trong python, ma trận liên thuộc có thể được biểu diễn bằng kiểu dữ liệu `list`

```
In [5]: # Thứ tự cạnh: 01,04,12,13,14,23,34
Inc = [[1, 1, 0, 0, 0, 0, 0],
       [1, 0, 1, 1, 1, 0, 0],
       [0, 0, 1, 0, 0, 1, 0],
       [0, 0, 0, 1, 0, 1, 1],
       [0, 1, 0, 0, 1, 0, 1]]
Inc
```

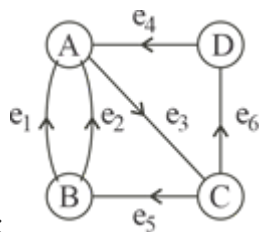
```
Out[5]: [[1, 1, 0, 0, 0, 0, 0],
         [1, 0, 1, 1, 1, 0, 0],
         [0, 0, 1, 0, 0, 1, 0],
         [0, 0, 0, 1, 0, 1, 1],
         [0, 1, 0, 0, 1, 0, 1]]
```

Hoặc sử dụng lớp `array` trong thư viện `numpy`

```
In [6]: import numpy as np
graph = np.array([[1, 1, 0, 0, 0, 0, 0],
                  [1, 0, 1, 1, 1, 0, 0],
                  [0, 0, 1, 0, 0, 1, 0],
                  [0, 0, 0, 1, 0, 1, 1],
                  [0, 1, 0, 0, 1, 0, 1]])
```

graph])

Out[6]: array([[1, 1, 0, 0, 0, 0, 0],
[1, 0, 1, 1, 1, 0, 0],
[0, 0, 1, 0, 0, 1, 0],
[0, 0, 0, 1, 0, 1, 1],
[0, 1, 0, 0, 1, 0, 1]])

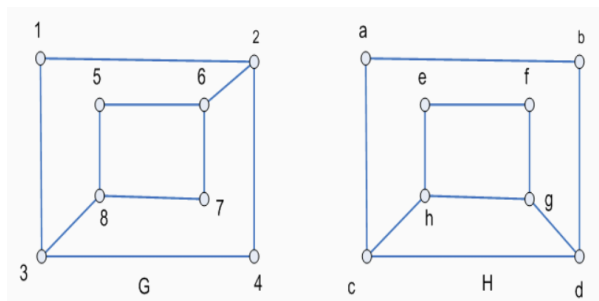


Ví dụ với đồ thị có hướng:

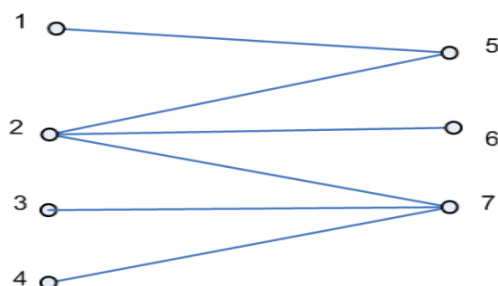
$$I(G) = \begin{pmatrix} -1 & -1 & 1 & -1 & 0 & 0 \\ 1 & 1 & 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & -1 \end{pmatrix}$$

1.4. Các đồ thị đặc biệt

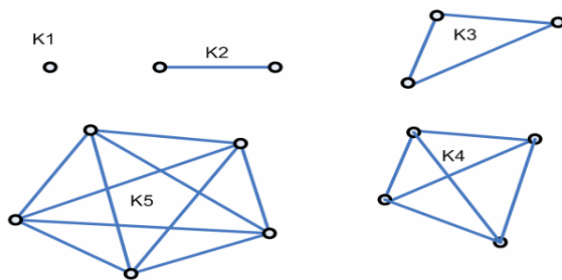
- **Đồ thị con** (subgraph): đồ thị con $G' = (V', E')$ của đồ thị $G = (V, E)$ nếu $V' \subseteq V, E' \subseteq E$.
- **Đồ thị đẳng cấu** (isomorphic graph): Hai đồ thị G_1 và G_2 được gọi là đẳng cấu với nhau, ký hiệu $G_1 \cong G_2$, nếu có thể vẽ lại (bằng cách dời đỉnh, dời cạnh,...) sao cho hai đồ thị này có hình vẽ y hệt nhau, dựa trên cơ sở sự hoán vị đỉnh và hoán vị cạnh.



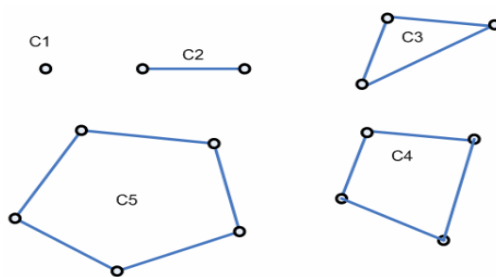
- **Đồ thị lưỡng phân** (bipartite graph): một đồ thị được gọi là lưỡng phân nếu nó có thể chia tập đỉnh thành 2 tập con rời nhau hoàn toàn:
 - $V_1 \neq \emptyset \neq V_2, V_1 \cap V_2 = \emptyset, V_1 \cup V_2 = V$
 - Trong từng đồ thị con V_1 và V_2 : không có cạnh nối 2 đỉnh bất kỳ



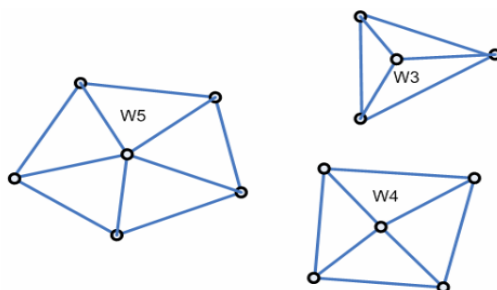
- **Đồ thị bù** (complement graph): đồ thị bù của đồ thị $G = (V, E)$ là $\overline{G} = (V, \overline{E})$ chứa tất cả các cạnh không có trong G .
- **Đồ thị đủ** (complete graph): là đơn đồ thị vô hướng, trong đó các đỉnh đôi một nối nhau. Khi đó, $\deg(A) = n - 1$, $|E| = nC2$.



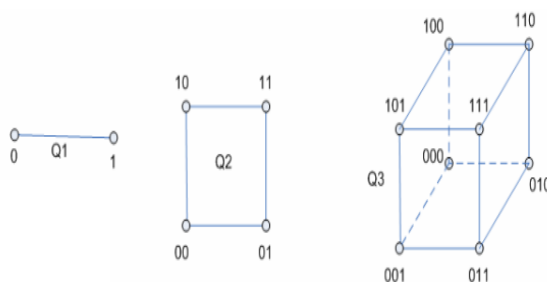
- **Đồ thị vòng**: là đơn đồ thị vô hướng có duy nhất một chu trình đơn đi qua tất cả các đỉnh.



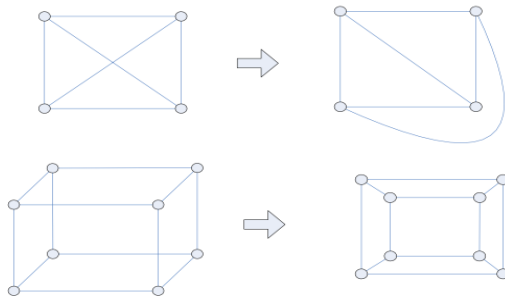
- **Đồ thị bánh xe**:



- **Đồ thị siêu khối**: là đồ thị có các đỉnh được đánh số bằng các chuỗi nhị phân độ dài n , có 2^n đỉnh và $n \cdot 2^{n-1}$ cạnh. Trong đó, 2 đỉnh là kề nhau nếu 2 chuỗi nhị phân tương ứng chỉ khác nhau 1 bit.



- **Đồ thị phẳng**: là đồ thị có thể vẽ trên 1 mặt phẳng mà các cạnh không giao nhau:



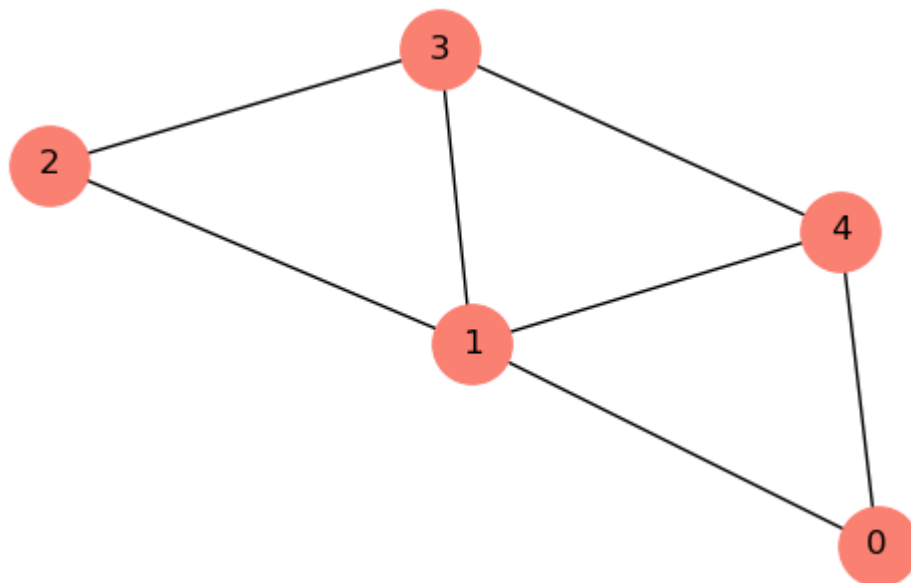
1.5. Vẽ đồ thị

Trong python, ta có thể vẽ đồ thị thông qua thư viện `networkx` và `matplotlib`

```
In [7]: import networkx as nx
import matplotlib.pyplot as plt

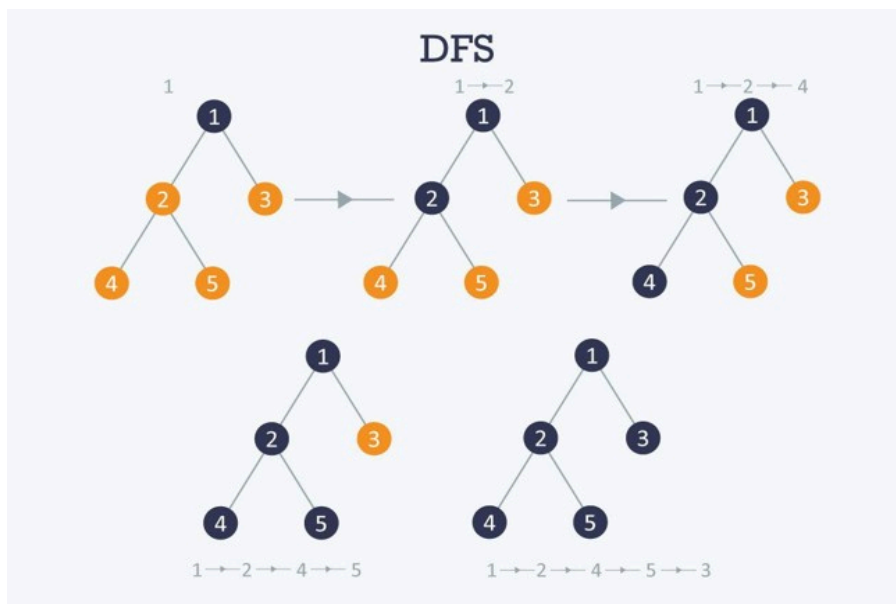
G = nx.Graph()
G.add_edges_from([(0,1),(0,4),
                  (1,2),(1,3),(1,4),
                  (2,3),
                  (3,4)
                  ])

plt.figure(figsize=(5,3))
nx.draw(G, with_labels=True, node_size=800, node_color='salmon')
plt.show()
```



2. Các thuật toán trên đồ thị

2.1. Thuật toán duyệt đồ thị theo chiều sâu Depth First Search (DFS)



Nguyên lý:

- Bắt đầu từ một đỉnh V bất kỳ của đồ thị.
- Chọn U là một đỉnh tùy ý kề với V .
- Lặp lại quá trình này với đỉnh U cho tới khi không tìm được đỉnh kề tiếp theo nữa thì trở về đỉnh ngay trước đỉnh mà không thể đi tiếp để tiếp tục tìm qua nhánh khác.

Minh họa:

- <https://www.cs.usfca.edu/~galles/visualization/DFS.html>
- <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>

Thuật toán:

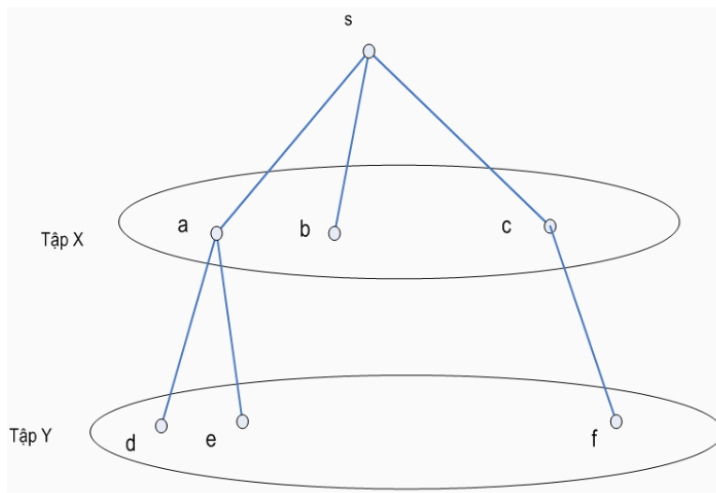
- Bước 1: Chọn V là 1 đỉnh của đồ thị để bắt đầu.
- Bước 2: Đặt V vào STACK.
- Bước 3: Lặp nếu STACK chưa rỗng:
 - Bước 3.1: Lấy đỉnh V_{last} từ STACK.
 - Bước 3.2: Duyệt đỉnh V_{last} : Nếu V_{last} chưa được thăm:
 - Thăm V_{last} .
 - Đặt các đỉnh kề chưa được thăm của V_{last} vào STACK.
- Bước 4: Kết thúc.

Trong python, sử dụng thư viện `networkx` để duyệt đồ thị bằng thuật toán DFS.

```
In [8]: dfs = list(nx.dfs_preorder_nodes(G, source=0))
print("DFS traversal:", dfs)
```

DFS traversal: [0, 1, 2, 3, 4]

2.2. Thuật toán duyệt đồ thị theo chiều rộng Breadth First Search (BFS)



Nguyên lý:

- Bắt đầu từ một đỉnh V bất kỳ của đồ thị.
- Duyệt tất cả những đỉnh kề với V , lưu vào tập S .
- Lần lượt xét các đỉnh thuộc S , lặp lại quá trình duyệt như với đỉnh V .

Minh họa:

- <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
- <https://www.cs.usfca.edu/~galles/visualization/BFS.html>

Thuật toán:

- Bước 1: Chọn V là 1 đỉnh của đồ thị để bắt đầu.
- Bước 2: Đặt V vào QUEUE. Thăm V .
- Bước 3: Lặp nếu QUEUE chưa rỗng:
 - Bước 3.1: Lấy đỉnh V_{first} từ QUEUE.
 - Bước 3.2: Duyệt đỉnh V_{first} :
 - Đặt các đỉnh kề chưa được thăm của V_{first} vào QUEUE.
 - Thăm chúng.
- Bước 4: Kết thúc.

Trong python, sử dụng thư viện `networkx` để duyệt đồ thị bằng thuật toán BFS.

```
In [9]: bfs = list(nx.bfs_tree(G, source=0).nodes())
print("BFS traversal:", bfs)
```

BFS traversal: [0, 1, 4, 2, 3]

2.3. Ứng dụng của thuật toán BFS và DFS:

Thuật toán BFS và DFS thường được sử dụng để:

- Tìm đường đi ngắn nhất/ dài nhất giữa 2 đỉnh tùy ý trong đồ thị không có trọng số,
- Kiểm tra tính liên thông, tìm số thành phần liên thông,
- Xác định cạnh cầu, đỉnh khớp,
- Tìm cây khung,
- Phát hiện chu trình trong đồ thị có hướng/ vô hướng,
- Kiểm tra đồ thị hai phía,

- v.v...

2.3.1. Tìm đường đi

- Dây chuyền (chain) : là một dãy luân phiên các đỉnh/ cạnh của đồ thị.
- Đường đi (path) : là một dây chuyền, bắt đầu ở một đỉnh và kết thúc ở một đỉnh.
- Đường đi đơn (simple path): khi tất cả các đỉnh và các cạnh trong đường đi là khác nhau đôi một (không lặp đỉnh, không lặp cạnh).

Định lý: Xét ma trận kề M thì $M^k[i][j]$ = số dây chuyền độ dài k từ đỉnh i đến đỉnh j .

```
In [10]: G1 = nx.Graph()
G1.add_edges_from([('A','B'), ('A','C'), ('A','D'),
                  ('B','C'),
                  ('E','F'), ('E','G'),
                  ('F','G')
                  ])
nx.has_path(G1, 'B', 'D'), nx.has_path(G1, 'E', 'A')
```

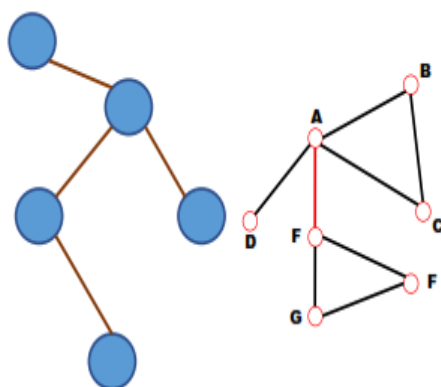
```
Out[10]: (True, False)
```

```
In [11]: list(nx.all_simple_paths(G, source=0, target=3))
```

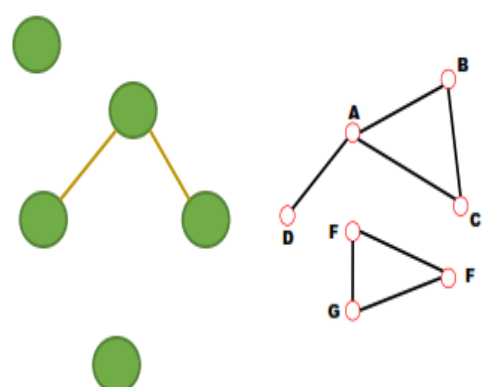
```
Out[11]: [[0, 1, 2, 3],
          [0, 1, 3],
          [0, 1, 4, 3],
          [0, 4, 1, 2, 3],
          [0, 4, 1, 3],
          [0, 4, 3]]
```

2.3.2. Liên thông

- Đồ thị được gọi là **liên thông** nếu luôn tìm được đường đi giữa 2 đỉnh bất kỳ trong đồ thị.
- Đồ thị **không liên thông**: là đồ thị chứa ít nhất hai đỉnh không liên thông.



connected graph



disconnected graph

- Thành phần liên thông: là một đồ thị con liên thông của đồ thị ban đầu. Đồ thị không liên thông sẽ có nhiều hơn 1 thành phần liên thông.
- Đồ thị có hướng $G = (V, E)$ được gọi là **liên thông yếu** (weakly connected) khi đồ thị vô hướng tương ứng với nó (tức đồ thị nhận được bằng cách thay các cạnh có hướng thành cạnh vô hướng) là liên thông.

- Đồ thị có hướng $G = (V, E)$ được gọi là **liên thông mạnh** (strongly connected) khi ứng với mọi cặp đỉnh, tồn tại một đường đi từ đỉnh này đến đỉnh kia và ngược lại, tồn tại một đường đi từ đỉnh kia đến đỉnh này.

```
In [12]: nx.is_connected(G)
```

```
Out[12]: True
```

```
In [13]: list(nx.connected_components(G))
```

```
Out[13]: [{0, 1, 2, 3, 4}]
```

```
In [14]: G1 = nx.Graph()
G1.add_edges_from([('A', 'B'), ('A', 'C'), ('A', 'D'),
                  ('B', 'C'),
                  ('E', 'F'), ('E', 'G'),
                  ('F', 'G')
                  ])
nx.is_connected(G1)
```

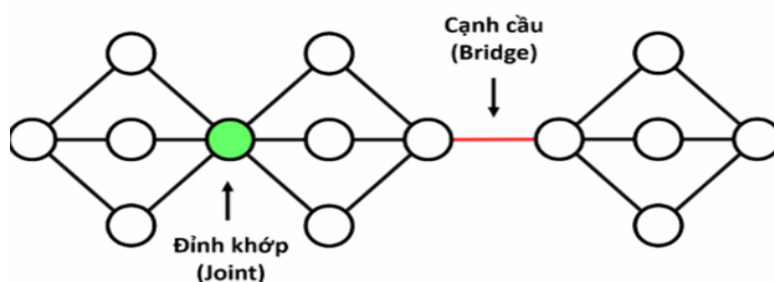
```
Out[14]: False
```

```
In [15]: list(nx.connected_components(G))
```

```
Out[15]: [{0, 1, 2, 3, 4}]
```

2.3.3. Cạnh cầu, đỉnh khớp

- Cạnh cầu (bridge): là cạnh mà nếu bỏ cạnh đó đi thì số thành phần liên thông trong đồ thị sẽ tăng lên.
- Đỉnh khớp (articulation point/ cut vertex): là đỉnh mà nếu bỏ đỉnh đó đi thì số thành phần liên thông trong đồ thị sẽ tăng lên.

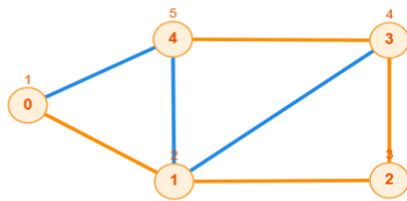


Cách tiếp cận:

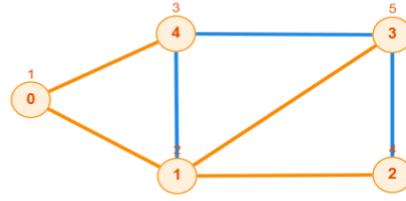
- Naive approach
- Sử dụng thuật toán Tarjan

2.3.4. Cây khung

- Cây khung (spanning tree): là một đồ thị con chứa tất cả các đỉnh của đồ thị gốc và là một cây (liên thông, không có chu trình).



DFS tree



BFS tree

- Tính chất:
 - Số cạnh của cây khung = số đỉnh - 1
 - Nếu đồ thị không liên thông thì không tồn tại cây khung
 - Một đồ thị liên thông có thể có nhiều cây khung khác nhau

2.3.5. Phát hiện chu trình

- DFS là phương pháp phổ biến nhất để phát hiện chu trình.
- Ý tưởng: khi DFS, nếu gặp một đỉnh đã thăm mà không phải đỉnh cha trực tiếp, thì tồn tại chu trình.
- BFS cũng được sử dụng để phát hiện chu trình.

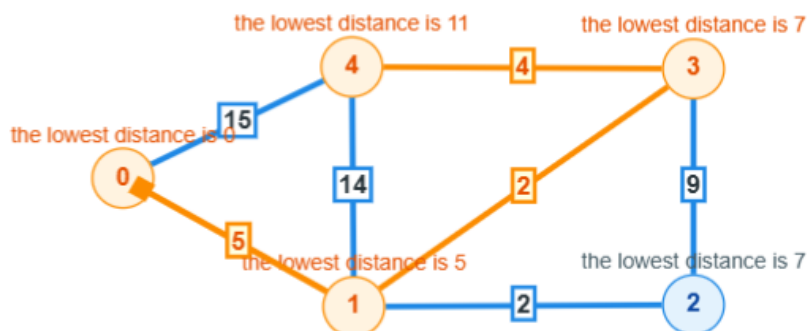
Minh họa: <https://www.geeksforgeeks.org/dsa/detect-cycle-undirected-graph/>

2.3.6. Kiểm tra đồ thị hai phía

Ý tưởng:

- BFS, DFS được sử dụng để "tô màu" các đỉnh của đồ thị bằng hai màu (ví dụ: 0 và 1).
- BFS: bắt đầu từ một đỉnh bất kỳ, ta gán màu cho nó, sau đó duyệt các đỉnh kề và gán cho chúng màu đối lập. Quá trình này tiếp tục theo từng lớp.
 - Nếu trong quá trình duyệt, ta gặp một cạnh nối hai đỉnh có cùng màu, thì đồ thị không phải là đồ thị hai phía.
 - Ngược lại, nếu việc tô màu hoàn tất mà không có xung đột, đồ thị là hai phía.
- DFS: tương tự BFS, nhưng theo hướng "đi sâu". Khi duyệt đến một đỉnh mới, ta gán màu ngược với đỉnh cha.
 - Nếu phát hiện một cạnh nối hai đỉnh cùng màu, thì kết luận đồ thị không phải là hai phía.

2.4. Thuật toán Dijkstra



Mục tiêu: Tìm đường đi ngắn nhất từ một đỉnh nguồn (Source) đến tất cả các đỉnh còn lại trong đồ thị có trọng số không âm.

Minh họa: <https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>

Thuật toán:

- Bước 1: Đánh dấu tất cả các đỉnh là chưa thăm
- Bước 2: Khởi tạo:
 - Khoảng cách đến đỉnh nguồn = 0,
 - Khoảng cách đến các đỉnh khác = ∞ ,
 - Tập đỉnh đã thăm = $\emptyset$
- Bước 3: Lặp nếu vẫn còn đỉnh chưa thăm:
 - Bước 3.1: Chọn đỉnh u chưa thăm có $dist[u]$ nhỏ nhất.
 - Bước 3.2: Đánh dấu u đã thăm.
 - Bước 3.3: Với mỗi đỉnh kề v của u nếu $dist[u] + weight(u, v) < dist[v]$ thì cập nhật $dist[v]$.

Công thức cập nhật: $dist[v] = \min(dist[v], dist[u] + weight(u, v))$.

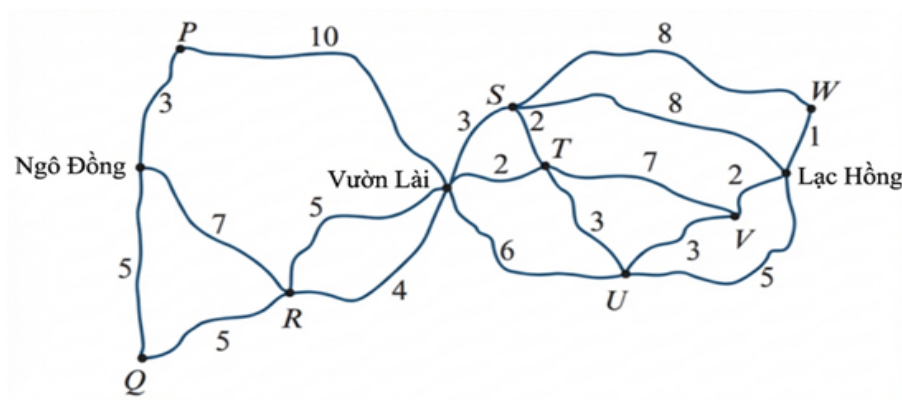
Pseudocode:

```
Dijkstra(Graph, source):
    dist[] = {∞}
    dist[source] = 0
    visited[] = {false}

    for i = 0 .. |V|-1:
        u = pick vertex with min dist not in visited
        visited[u] = true

        for v in neighbors of u:
            if dist[u] + weight(u, v) < dist[v]:
                dist[v] = dist[u] + weight(u, v)
```

Trong python, sử dụng thư viện `networkx` để tìm đường đi ngắn nhất theo thuật toán Dijkstra:



```
In [16]: import pandas as pd
dijkstra = pd.read_excel('graph_data_path.xlsx', sheet_name='Dijk_stepbystep')
dijkstra
```

Out[16]:

	Step	VL	S	T	U	V	W	LH
0	0	0	(¥, -)	(¥, -)	(¥, -)	(¥, -)	(¥, -)	(¥, -)
1	1	-	(3, VL)	(2, VL)	(6, VL)	(¥, -)	(¥, -)	(¥, -)
2	2	-	(3, VL)	-	(5, T)	(9, T)	(¥, -)	(¥, -)
3	3	-	-	-	(5, T)	(9, T)	(11, S)	(11, S)
4	4	-	-	-	-	(8, U)	(11, S)	(10, U)
5	5	-	-	-	-	-	(11, S)	(10, U)
6	6	-	-	-	-	-	(11, S)	-

In [17]:

```
import networkx as nx

data_VL_LH = pd.read_excel('graph_data_path.xlsx', sheet_name='Dijk')
G2 = nx.Graph()
G2.add_weighted_edges_from(
    [(row['from'], row['to'], row['weight']) for _, row in
     data_VL_LH.iterrows()])

total_weight, path = nx.single_source_dijkstra(G2, source='VL', target='LH', weight='weight')
print("Shortest path from Vườn Lài to Lạc Hồng:", path)
print("Distance:", total_weight)
```

Shortest path from Vườn Lài to Lạc Hồng: ['VL', 'T', 'U', 'LH']
Distance: 10

2.5. Thuật toán Bellman–Ford

Mục tiêu: Tìm đường đi ngắn nhất từ một đỉnh nguồn (Source) đến tất cả các đỉnh còn lại trong đồ thị có trọng số (cho phép trọng số âm).

Minh họa: <https://www.geeksforgeeks.org/dsa/bellman-ford-algorithm-dp-23/>

Pseudocode:

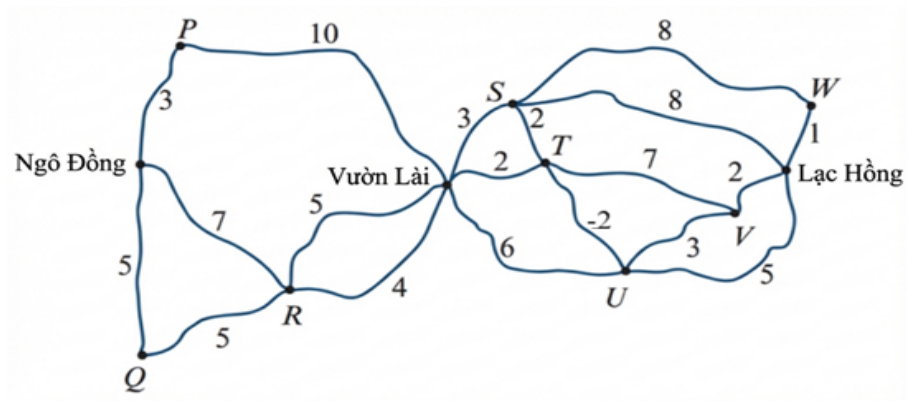
```
Bellman_Ford(Graph, source):
    for v in V:
        dist[v] = ∞
        parent[v] = None

    dist[source] = 0

    for i = 1 .. |V| - 1:
        for (u, v) in E:
            if dist[u] + weight(u, v) < dist[v]:
                dist[v] = dist[u] + weight(u, v)
                parent[v] = u

    for (u, v) in E:
        if dist[u] + weight(u, v) < dist[v]:
            return "Negative cycle detected."

    return dist, parent
```



Trong python, sử dụng thư viện `networkx` để tìm đường đi ngắn nhất theo thuật toán Bellman-Ford:

```
In [18]: import pandas as pd
import networkx as nx
data_VLLH = pd.read_excel('graph_data_path.xlsx', sheet_name='Bell')
data_VLLH
```

Out[18]:

	from	to	weight
0	ND	P	3
1	ND	Q	5
2	ND	R	7
3	P	VL	10
4	Q	R	5
5	R	VL	5
6	R	VL	4
7	VL	S	3
8	VL	T	2
9	VL	U	6
10	S	W	8
11	S	LH	8
12	S	T	2
13	T	U	-2
14	T	V	7
15	U	V	3
16	U	LH	5
17	V	LH	2
18	W	LH	1

```
In [19]: # G3 = nx.Graph()
# G3.add_weighted_edges_from(
#         [(row['from'], row['to'], row['weight']) for _, row in data_VLLH.iterrows()]
#     )
# total_weight3, path3 = nx.single_source_bellman_ford(G3, source='VL', target='LH')
```

```
# print("Shortest path from Vườn Lài to Lạc Hồng:", path3)
# print("Distance:", total_weight3)
```

Nếu đồ thị trên có hướng với mỗi cung có thứ tự đỉnh từ **from** đến **to** thì:

```
In [20]: G4 = nx.DiGraph()
G4.add_weighted_edges_from(
    [(row['from'], row['to'], row['weight']) for _, row in
     ])
total_weight4, path4 = nx.single_source_bellman_ford(G4, source='VL', target='LH',
print("Shortest path from Vườn Lài to Lạc Hồng:", path4)
print("Distance:", total_weight4)
```

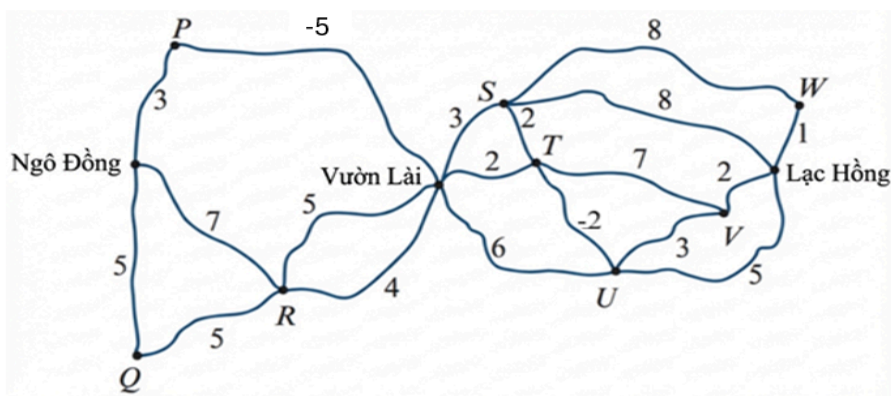
Shortest path from Vườn Lài to Lạc Hồng: ['VL', 'T', 'U', 'LH']
Distance: 5

```
In [21]: import pandas as pd
bellman = pd.read_excel('graph_data_path.xlsx', sheet_name='Bell_stepbystep')
bellman
```

```
Out[21]:
```

	Step	VL	S	T	U	V	W	LH
0	0	0	(¥, -)	(¥, -)	(¥, -)	(¥, -)	(¥, -)	(¥, -)
1	1	0	(3, VL)	(2, VL)	(6, VL)	(¥, -)	(¥, -)	(¥, -)
2	2	0	(3, VL)	(2, VL)	(0, T)	(9, T)	(11, S)	(11, S)
3	3	0	(3, VL)	(2, VL)	(0, T)	(3, U)	(11, S)	(5, U)
4	4	0	(3, VL)	(2, VL)	(0, T)	(3, U)	(11, S)	(5, U)

Thay đổi đồ thị trên với cung (P → Vườn Lài) có trọng số là -5 thì:



```
In [22]: data_NDV1 = pd.read_excel('graph_data_path.xlsx', sheet_name='Bell2')
data_NDV1
```

Out[22]:

	from	to	weight
0	ND	P	3
1	ND	Q	5
2	ND	R	7
3	P	VL	-5
4	Q	R	5
5	R	VL	5
6	R	VL	4

```
In [23]: G5 = nx.DiGraph()
G5.add_weighted_edges_from(
    [(row['from'], row['to'], row['weight']) for _, row in
     ])

total_weight5, path5 = nx.single_source_bellman_ford(G5, source='ND', target='VL',
print("Shortest path from Ngõ Đồng to Vườn Lài:", path5)
print("Distance:", total_weight5)
```

Shortest path from Ngõ Đồng to Vườn Lài: ['ND', 'P', 'VL']
Distance: -2

2.6. Thuật toán Floyd Warshall

Mục tiêu: Tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh trong đồ thị có trọng số (cho phép trọng số âm).

Minh họa: <https://www.geeksforgeeks.org/dsa/floyd-warshall-algorithm-dp-16/>

Pseudocode:

```
FLOYD_WARSHALL(G):
    for i in V:
        for j in V:
            if i == j:
                D[i][j] = 0
            else if có cạnh (i, j):
                D[i][j] = weight(i, j)
            else:
                D[i][j] = ∞

    for k in V:
        for i in V:
            for j in V:
                if D[i][j] > D[i][k] + D[k][j]:
                    D[i][j] = D[i][k] + D[k][j]

    return D
```

Trong python, sử dụng thư viện `networkx` để tìm đường đi ngắn nhất theo thuật toán Floyd Warshall:

```
In [24]: pred, dist = nx.floyd_warshall_predecessor_and_distance(G5, weight='weight')
```

```
def get_path(pred, source, destination):
    path = [destination]
    while destination != source:
        destination = pred[source][destination]
        path.append(destination)
    return path.reverse()

print("Shortest path from Ngô Đồng to Vườn Lài:", get_path(pred, 'ND', 'VL'))
print("Distance:", dist['ND']['VL'])
```

Shortest path from Ngô Đồng to Vườn Lài: None
Distance: -2

```
In [25]: df = pd.DataFrame(dist)
df = df.sort_index().sort_index(axis=0)
df
```

```
Out[25]:
```

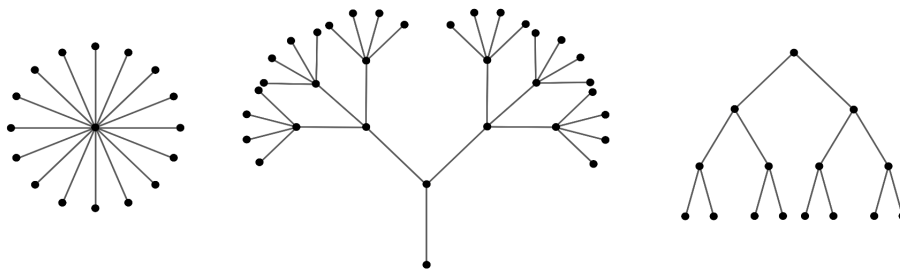
	ND	P	Q	R	VL
ND	0	inf	inf	inf	inf
P	3	0.0	inf	inf	inf
Q	5	inf	0.0	inf	inf
R	7	inf	5.0	0.0	inf
VL	-2	-5.0	9.0	4.0	0.0

3. Cây và các thuật toán trên cây

3.1. Tổng quan về cây

Cây T là một đơn đồ thị $G = (V, E)$ thỏa mãn các đặc điểm sau đây:

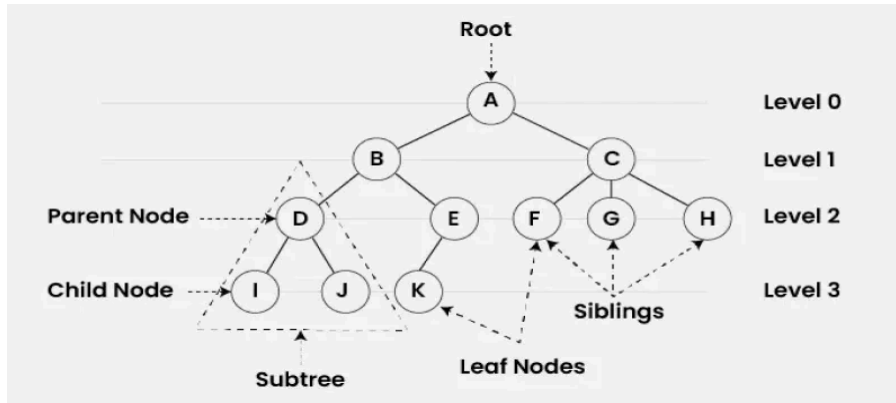
- vô hướng có n đỉnh và $n - 1$ cạnh,
- liên thông,
- không có chu trình.



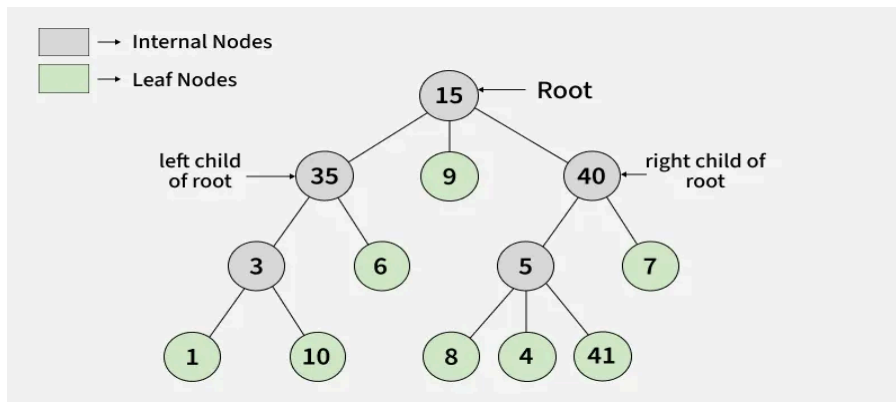
Các thuật ngữ cơ bản:

- node (đỉnh/ nút): là phần tử của cây
- edge (cạnh): đường liên kết giữa hai node
- parent (of node n): là node trên trực tiếp của node n trong cây
- child (of node n): là node dưới trực tiếp của node n trong cây

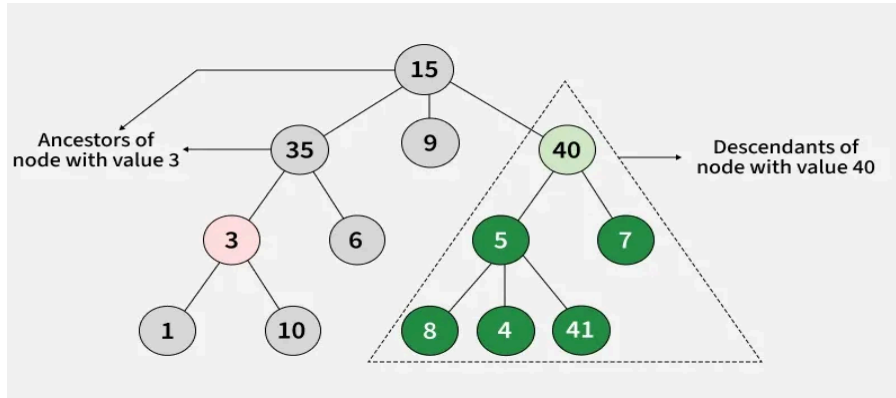
- root (gốc): là node duy nhất trong cây không có cha



- internal node (node trong): là node có ít nhất một con
- leaf node (external node, lá): là node không có con

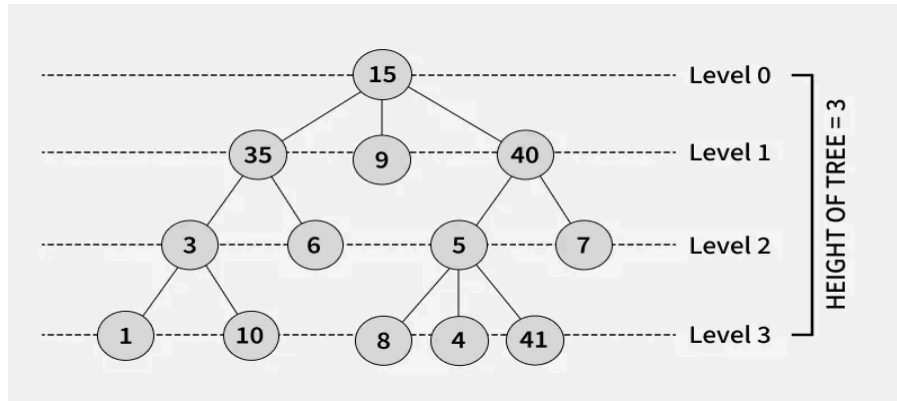


- ancestor (of node n): là một node thuộc đường từ root đến n
- descendant (of node n): là một node thuộc đường từ node n đến một lá



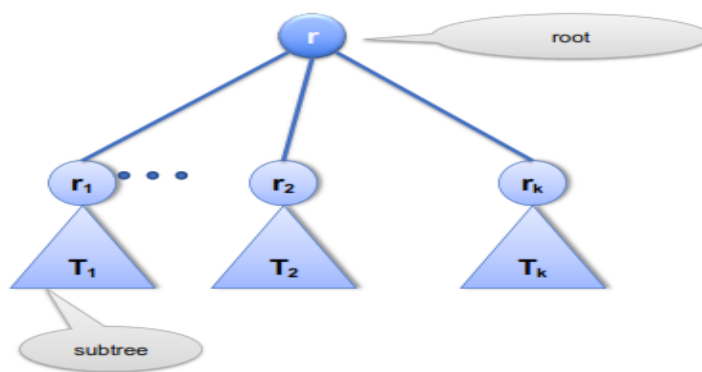
- degree/ order (bậc):
 - $degree(n)$ = số con của node n
 - $degree(T) = \max(degree(n)), n \in V$ (bậc lớn nhất của các node trong cây)
- depth/ level of a node (độ sâu/ mức của node):
 - $level(n) = 0$ nếu n là root
 - $level(n)$ = số cạnh từ root đến node n
- height (chiều cao):
 - $height(T) = -1$ nếu cây T là rỗng

- $height(T) = \max(level(n)), n \in V$ (độ sâu/ mức lớn nhất của cây)



Minh họa: <https://www.geeksforgeeks.org/dsa/introduction-to-tree-data-structure/>

Cây con (subtree) là một node bất kỳ trong cây cùng với tất cả hậu duệ của nó.



Cây khung (Spanning Tree): là một đồ thị con chứa tất cả các đỉnh của đồ thị ban đầu và là một cây (liên thông, không có chu trình).

- Cây khung tối thiểu (Minimum Spanning Tree - MST): là cây khung mà tổng trọng số các cạnh là nhỏ nhất.
- Cây khung tối đại (Maximum Spanning Tree - MaxST): là cây khung mà tổng trọng số các cạnh là lớn nhất.

Định lý: Cho $G = (V, E)$ là một đơn đồ thị vô hướng n đỉnh. Các phát biểu sau đây là tương đương:

- G là cây
- G liên thông và có $n - 1$ cạnh
- G không có chu trình và có $n - 1$ cạnh
- G liên thông và mỗi cạnh của nó đều là cầu
- Hai đỉnh bất kỳ của G được nối với nhau bởi đúng một đường đi duy nhất
- G không có chu trình nhưng nếu thêm một cạnh bất kỳ vào G thì sẽ tạo thành một chu trình

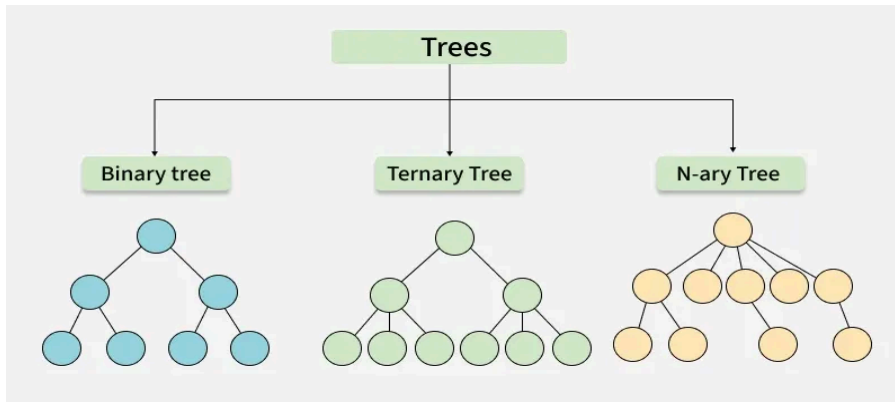
3.1.1. Các thuộc tính của cây

- Các thuộc tính về cạnh:
 - Số cạnh của cây có n node là: $|E| = n - 1$
 - Thêm 1 cạnh $\rightarrow$ tạo 1 chu trình
 - Xóa 1 cạnh $\rightarrow$ mất tính liên thông

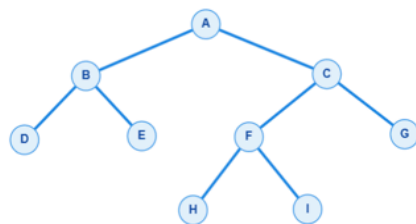
- Các thuộc tính về đỉnh/ node:
 - Mỗi node có 0 hoặc nhiều node con
 - Mỗi node có nhiều nhất 1 node cha. Root là node duy nhất không có node cha

3.1.2. Phân loại cây

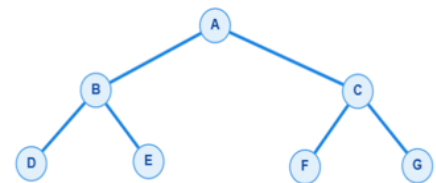
Dựa trên số con mà mỗi node có thể có.



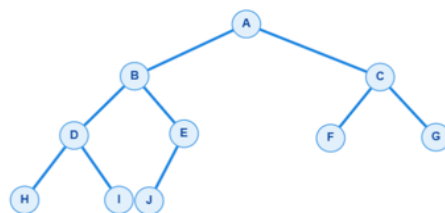
- Cây nhị phân (Binary Tree): mỗi node có tối đa 2 node con (trái, phải)
 - Cây nhị phân đầy đủ (Full Binary Tree): là cây nhị phân mà tất cả node trong đều có đúng 2 con
 - Cây nhị phân hoàn hảo (Perfect Binary Tree): là cây nhị phân **full** mà:
 - Tất cả lá đều có cùng level
 - Cây nhị phân hoàn chỉnh (Complete Binary Tree): là cây nhị phân chiều cao h mà:
 - Nó **perfect** đến mức $h - 1$
 - Ở mức cuối (mức h): các lá được lấp đầy từ trái đến sang phải (bên phải có thể khuyết)
 - Heap: là cây nhị phân **complete** có ràng buộc giữa các node như sau:
 - Giá trị node cha $\geq$ giá trị node con (Heap cực đại), hoặc:
 - Giá trị node cha $\leq$ giá trị node con (Heap cực tiểu)



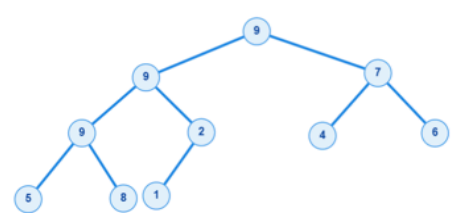
Full Binary Tree



Perfect Binary Tree



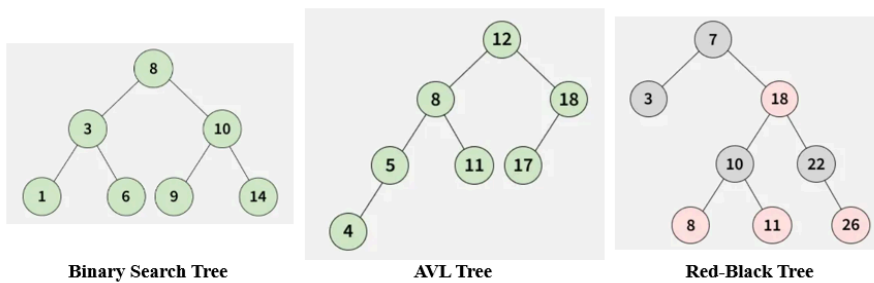
Complete Binary Tree



Heap

- Cây tam phân (Ternary Tree): mỗi node có tối đa 3 node con (trái, giữa, phải)
- Cây N-phân (N-ary Tree/ Generic Tree): mỗi node có tối đa N node con

3.1.3. Các cây đặc biệt



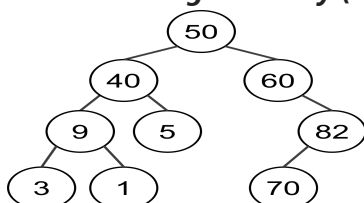
- **Cây nhị phân tìm kiếm (Binary Search Tree):** là cây nhị phân có ràng buộc như sau:
 - Tất cả giá trị node trong cây con trái < giá trị node cha < tất cả giá trị node trong cây con phải (left < root < right)
 - Cả T_{left} lẫn T_{right} đều là cây nhị phân tìm kiếm
- **Cây nhị phân cân bằng (Balanced Binary Tree):** là cây nhị phân mà ứng với mỗi node thì sự khác biệt về chiều cao giữa cây con trái và cây con phải là nhỏ nhất
- **Cây AVL (AVL Tree):** (Adelson, Velski và Landis) là cây nhị phân tìm kiếm tự cân bằng, ứng với mỗi node:

$$|height(left) - height(right)| \leq 1$$

- **Cây đỏ đen (Red-Black Tree):** là cây nhị phân tìm kiếm tự cân bằng có các ràng buộc sau:
 - Mỗi node có màu đỏ hoặc đen
 - Node root luôn đen
 - Một node đỏ không có cha đỏ và con đỏ
- **B-Tree** bậc m : là một cây tìm kiếm đa nhánh (multiway search tree)
 - Số con của root: $2 \leq \text{số con} \leq m$
 - Số con của node trong: $\lceil m/2 \rceil \leq \text{số con} \leq m$
 - Mỗi node trong có k con thì chứa $k - 1$ key
 - Tất cả lá đều có cùng level

3.1.4. Biểu diễn cây

Biểu diễn bằng sơ đồ cây (Visual Tree Drawing):



Biểu diễn bằng danh sách đỉnh kề (danh sách con trực tiếp - Adjacency List):

Trong python, danh sách kề có thể được biểu diễn bằng kiểu dữ liệu `dictionary`. Trong đó, mỗi node lưu danh sách các node con trực tiếp.

```
In [26]: tree = {50: [40, 60],
                40: [9, 5],
                60: [82],
                9: [3, 1],
                82: [70],
                5: [],
                3: [],
                1: []}
```

```

70: []
}

```

tree

```

Out[26]: {50: [40, 60],
40: [9, 5],
60: [82],
9: [3, 1],
82: [70],
5: [],
3: [],
1: [],
70: []}

```

Biểu diễn bằng danh sách cha (Parent Array):

Trong python, danh sách cha có thể được biểu diễn bằng kiểu dữ liệu `dictionary`. Trong đó, mỗi node lưu cha của nó.

```

In [27]: parent = {50: None,
40: 50,
60: 50,
9: 40,
5: 40,
82: 60,
3: 9,
1: 9,
70: 82}
parent

```

```

Out[27]: {50: None, 40: 50, 60: 50, 9: 40, 5: 40, 82: 60, 3: 9, 1: 9, 70: 82}

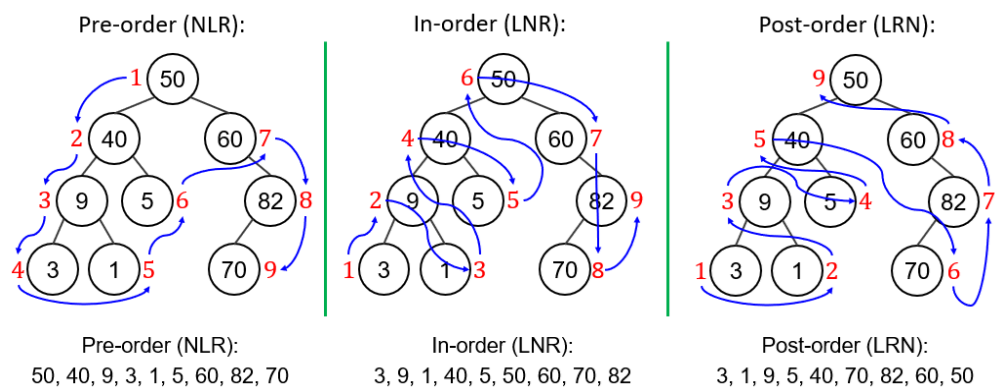
```

3.2. Các thuật toán trên cây

3.2.1. Thuật toán duyệt cây (Tree Traversal)

Có 3 thuật toán duyệt cây cơ bản:

- Pre-order: **Root** → Left → Right
- In-order: Left → **Root** → Right
- Post-order: Left → Right → **Root**



3.2.2. Thuật toán Prim

Mục tiêu: Tìm cây khung tối thiểu (Minimum Spanning Tree – MST) cho một đồ thị vô hướng, liên thông, có trọng số.

Ý tưởng: Bắt đầu từ một đỉnh, ta "nuôi" cây lớn dần bằng cách luôn chọn cạnh có trọng số nhỏ nhất nối từ một đỉnh đã nằm trong cây sang một đỉnh chưa nằm trong cây.

Minh hoạ:

- <https://www.cs.usfca.edu/~galles/visualization/Prim.html>
- <https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>

Pseudocode:

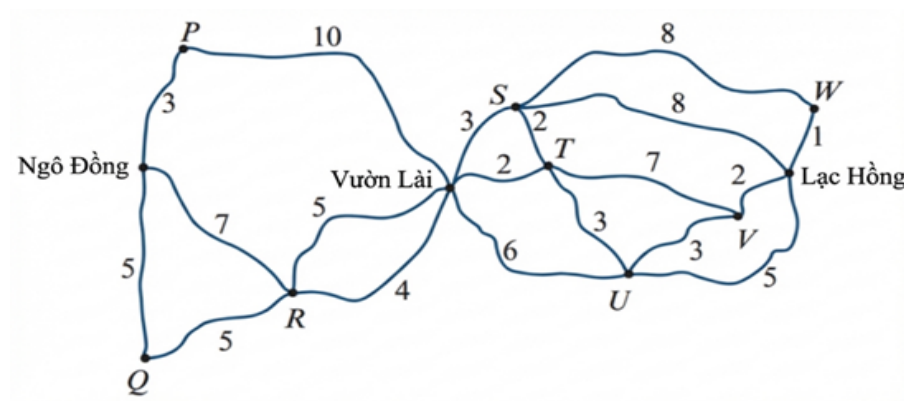
```

Prim(Graph, start_node):
    create key[] = {infinity}, mst_set[] = {false}, parent[] = {-1}
    key[start_node] = 0

    for count in 0..V-1:
        u = pick vertex with min key value not in mst_set
        mst_set[u] = true

        for v in neighbors of u:
            if graph[u][v] > 0 and v not in mst_set and
graph[u][v] < key[v]:
                parent[v] = u
                key[v] = graph[u][v]
    print MST using parent array

```



Step	Candidate Edges	Chosen Edge	Weight	Visited Nodes
0			0	NĐ
1	(NĐ,P), (NĐ,Q), (NĐ,R)	(NĐ,P)	3	NĐ,P
2	(NĐ,Q), (NĐ,R), (P,VL)	(NĐ,Q)	5	NĐ,P,Q
3	(NĐ,R), (P,VL), (Q,R)	(Q,R)	5	NĐ,P,Q,R
4	(NĐ,R), (P,VL), (R,VL)	(R,VL)	4	NĐ,P,Q,R,VL
			17	stop
0			0	VL
1	(VL,S,3), (VL,T,2), (VL,U,6)	(VL,T,2)	2	VL,T
2	(VL,S,3), (VL,U,6), (T,S,2), (T,U,3), (T,V,7)	(T,S,2)	2	VL,T,S
3	(VL,S,3), (VL,U,6), (T,U,3), (T,V,7), (S,W,8), (S,LH,8)	(T,U,3)	3	VL,T,S,U
4	(VL,U,6), (T,V,7), (S,W,8), (S,LH,8), (U,V,3), (U,LH,5)	(U,V,3)	3	VL,T,S,U,V
5	(VL,U,6), (T,V,7), (S,W,8), (S,LH,8), (U,LH,5), (V,LH,2)	(V,LH,2)	2	VL,T,S,U,V,LH
6	(VL,U,6), (T,V,7), (S,W,8), (S,LH,8), (U,LH,5), (LH,W,1)	(LH,W,1)	1	VL,T,S,U,V,LH,W
			13	stop

Trong python, sử dụng thư viện `networkx` để tìm cây khung tối thiểu cho đồ thị theo thuật toán Prim:

```
In [28]: import pandas as pd
import networkx as nx

data_VL_LH = pd.read_excel('graph_data_path.xlsx', sheet_name='Dijk')
G2 = nx.Graph()
G2.add_weighted_edges_from(
    [(row['from'], row['to'], row['weight']) for _, row in
     ])

# Tìm MST bằng Prim
mst_prim = nx.minimum_spanning_tree(G2, algorithm='prim')
maxst_prim = nx.maximum_spanning_tree(G2, algorithm='prim')

# In các cạnh trong MST:
print("Các cạnh trong MST theo Prim:")
for u, v, d in mst_prim.edges(data=True):
    print(f"{u}-{v}, weight = {d['weight']}")

# Tính tổng trọng số MST:
total_weight_mst_prim = sum(d['weight'] for u, v, d in mst_prim.edges(data=True))
print("Tổng trọng số MST theo Prim:", total_weight_mst_prim)

print('\n')
# In các cạnh trong MaxST:
print("Các cạnh trong MaxST theo Prim:")
for u, v, d in maxst_prim.edges(data=True):
    print(f"{u}-{v}, weight = {d['weight']}")

# Tính tổng trọng số MaxST:
total_weight_maxst_prim = sum(d['weight'] for u, v, d in maxst_prim.edges(data=True))
print("Tổng trọng số MaxST theo Prim:", total_weight_maxst_prim)
```

```
Các cạnh trong MST theo Prim:
ND-Q, weight = 5
ND-P, weight = 3
Q-R, weight = 5
R-VL, weight = 4
VL-T, weight = 2
S-T, weight = 2
T-U, weight = 3
U-V, weight = 3
W-LH, weight = 1
LH-V, weight = 2
Tổng trọng số MST theo Prim: 30
```

```
Các cạnh trong MaxST theo Prim:
ND-R, weight = 7
P-VL, weight = 10
Q-R, weight = 5
R-VL, weight = 4
VL-U, weight = 6
S-LH, weight = 8
S-W, weight = 8
T-U, weight = 3
T-V, weight = 7
U-LH, weight = 5
Tổng trọng số MaxST theo Prim: 63
```

3.2.3. Thuật toán Kruskal

Mục tiêu: Tìm cây khung tối thiểu (Minimum Spanning Tree – MST) cho một đồ thị vô hướng, liên thông, có trọng số.

Ý tưởng:

- Coi mỗi đỉnh là một cây riêng biệt.
- Sắp xếp tất cả các cạnh theo trọng số tăng dần. Lần lượt chọn cạnh nhỏ nhất, nếu cạnh đó:
 - nối hai cây riêng biệt thì hợp nhất chúng lại;
 - tạo thành chu trình thì bỏ qua.

Minh họa:

- <https://www.cs.usfca.edu/~galles/visualization/Kruskal.html>
- <https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>

Pseudocode:

```
Kruskal(Graph):  
    result = []  
    sort all edges by weight  
    create subsets for each vertex (Union-Find structure)  
  
    for each edge (u, v, weight) in sorted_edges:  
        x = find(u)  
        y = find(v)  
        if x != y:  
            result.add(edge)  
            union(x, y)
```

Index	1	2	3	4	5	6	7	8	9	10	11	12
Edge	(W,LH)	(VL,T)	(S,T)	(V,LH)	(ND,P)	(VL,S)	(T,U)	(U,V)	(R,VL)	(ND,Q)	(Q,R)	(R,VL)
weight	1	2	2	2	3	3	3	3	4	5	5	5
choose	☒	☒	☒	☒	☒	☐	☒	☒	☒	☒	☒	☐

step	ND	P	Q	R	VL	S	T	U	V	W	LH
0	ND	P	Q	R	VL	S	T	U	V	W	LH
1	ND	P	Q	R	VL	S	T	U	V	W	W
2	ND	P	Q	R	VL	S	VL	U	V	W	W
3	ND	P	Q	R	VL	VL	VL	U	V	W	W
4	ND	P	Q	R	VL	VL	VL	U	V	V	V
5	ND	ND	Q	R	VL	VL	VL	U	V	V	V
6	ND	ND	Q	R	VL	VL	VL	U	V	V	V
7	ND	ND	Q	R	VL	VL	VL	VL	V	V	V
8	ND	ND	Q	R	VL	VL	VL	VL	VL	VL	VL
9	ND	ND	Q	VL	VL	VL	VL	VL	VL	VL	VL
10	ND	ND	ND	VL	VL	VL	VL	VL	VL	VL	VL
11	ND	ND	ND	ND	ND	ND	ND	ND	ND	ND	ND

Trong python, sử dụng thư viện `networkx` để tìm cây khung tối thiểu cho đồ thị theo thuật toán Kruskal:

```
In [29]: import pandas as pd  
import networkx as nx  
import matplotlib.pyplot as plt  
  
data_VL_LH = pd.read_excel('graph_data_path.xlsx', sheet_name='Dijk')  
G2 = nx.Graph()  
G2.add_weighted_edges_from(
```

```

        [(row['from'], row['to'], row['weight']) for _, row in
        )

# Tìm MST bằng Kruskal
mst_kruskal = nx.minimum_spanning_tree(G2, algorithm='kruskal')
maxst_kruskal = nx.maximum_spanning_tree(G2, algorithm='kruskal')

# Vẽ cây khung
pos = nx.spring_layout(G2)
nx.draw(G2, pos, with_labels=True, node_color='lightgray')
nx.draw_networkx_edges(mst_kruskal, pos, edge_color='orange', width=2)
plt.show()

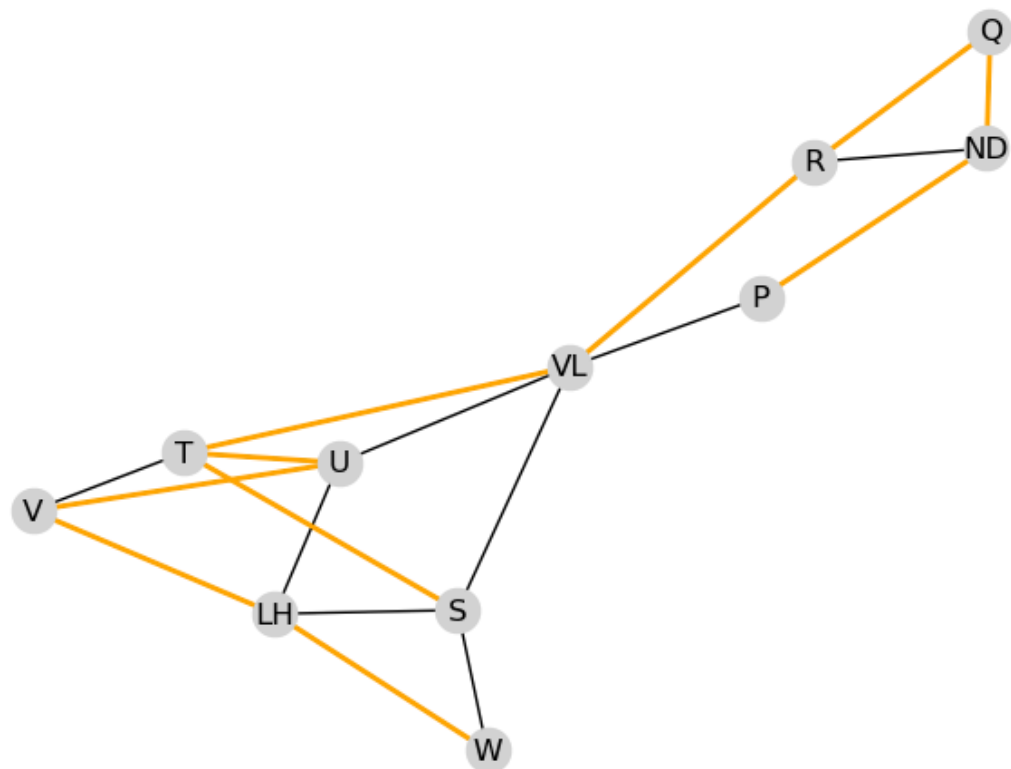
# In các cạnh trong MST:
print("Các cạnh trong MST theo kruskal:")
for u, v, d in mst_kruskal.edges(data=True):
    print(f"{u}-{v}, weight = {d['weight']}")

# Tính tổng trọng số MST:
total_weight_mst_kruskal = sum(d['weight'] for u, v, d in mst_kruskal.edges(data=True))
print("Tổng trọng số MST theo Kruskal:", total_weight_mst_kruskal)

print('\n')
# In các cạnh trong MaxST:
print("Các cạnh trong MaxST theo kruskal:")
for u, v, d in maxst_kruskal.edges(data=True):
    print(f"{u}-{v}, weight = {d['weight']}")

# Tính tổng trọng số MaxST:
total_weight_maxst_kruskal = sum(d['weight'] for u, v, d in maxst_kruskal.edges(data=True))
print("Tổng trọng số MaxST theo kruskal:", total_weight_maxst_kruskal)

```



Các cạnh trong MST theo kruskal:
ND-Q, weight = 5
ND-P, weight = 3
Q-R, weight = 5
R-VL, weight = 4
VL-T, weight = 2
S-T, weight = 2
T-U, weight = 3
U-V, weight = 3
W-LH, weight = 1
LH-V, weight = 2
Tổng trọng số MST theo Kruskal: 30

Các cạnh trong MaxST theo kruskal:
ND-R, weight = 7
ND-Q, weight = 5
P-VL, weight = 10
R-VL, weight = 4
VL-U, weight = 6
S-W, weight = 8
S-LH, weight = 8
T-V, weight = 7
T-U, weight = 3
U-LH, weight = 5
Tổng trọng số MaxST theo kruskal: 63

3.2.4. Ứng dụng của thuật toán Prim và Kruskal

Ứng dụng:

- Tìm cây khung tối thiểu
- Tìm cây khung tối đại

Áp dụng trong thực tế:

- Thiết kế hạ tầng mạng (Network Design) điện/ internet/ cáp quang/ nước,...
- Thiết kế hệ thống đường bộ/ đường sắt/ đường ống vận chuyển,...
- Thiết kế hệ thống camera an ninh.
- v.v...

4. Luồng trong mạng (Network flows)

4.1. Tổng quan

Trong lý thuyết đồ thị, **Network Flows** (luồng trong mạng) là mô hình biểu diễn việc luân chuyển một đại lượng nào đó (nước, khí, dữ liệu, hàng hóa,...) qua một mạng lưới (network).

Mạng (Network) là một đồ thị có hướng $G = (V, E)$, trong đó:

- Có duy nhất một đỉnh không có cung đi vào, được gọi là **đỉnh phát** s (source)
- Có duy nhất một đỉnh không có cung đi ra, được gọi là **đỉnh thu** t (sink)
- Mỗi cung (u, v) được gán một số nguyên không âm $c(u, v)$, gọi là **khả năng thông qua** (capacity) của cung đó

Quy ước: nếu cung (u, v) không tồn tại thì $capacity(u, v) = 0$

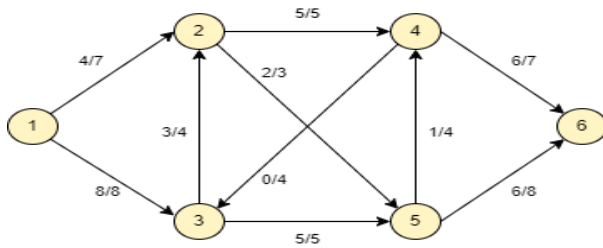
Luồng (Flow) $s - t$ trong mạng $G = (V, E)$ là một ánh xạ $f : E \rightarrow \mathbb{R}^+$ thỏa mãn:

- Ràng buộc thông lượng (Capacity constraint): $0 \leq f(u, v) \leq c(u, v)$
- Điều kiện cân bằng luồng (Flow conservation): tại mỗi đỉnh trung gian, tổng giá trị thông lượng trên cung vào bằng tổng giá trị thông lượng trên cung ra

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w), \forall v \in V \setminus \{s, t\}$$

Tải trọng của luồng = tổng giá trị thông lượng trên các cung đi ra từ đỉnh phát = tổng giá trị thông lượng trên các cung đi vào đỉnh thu

$$val(f) = \sum_{v \in V} f(s, v) = \sum_{u \in V} f(u, t)$$



Ghi chú: Số trên mỗi cung tương ứng với flow/capacity của cung đó

4.2. Bài toán Max-Flow

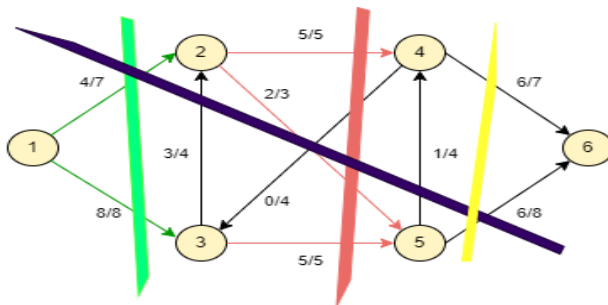
Luồng cực đại (Maximum flow) là luồng f^* trong mạng $G = (V, E)$ có tải trọng luồng, $val(f^*)$, là lớn nhất trong số tất cả các luồng f trong mạng.

Bài toán Max-Flow là bài toán đi tìm luồng cực đại trong mạng.

Lát cắt: là một cách phân hoạch tập đỉnh V thành 2 tập hợp con rời nhau, trong đó một tập chứa source và một tập chứa sink.

Khả năng thông qua của lát cắt $(X|Y)$ là: $c(X|Y) = \sum c(u, v), u \in X, v \in Y$.

Lát cắt nhỏ nhất (Minimum cut) là lát cắt có khả năng thông qua nhỏ nhất trong số tất cả các lát cắt.



Định lý: $val(f) \leq c(X|Y)$, với f là một luồng trong mạng và $(X|Y)$ là một lát cắt nào đó của mạng.

Định lý Max flow - Min cut: Trong một mạng bất kỳ, giá trị/ tải trọng của luồng cực đại luôn bằng khả năng thông qua của lát cắt nhỏ nhất:

$$val(\text{max flow}) = capacity(\text{lát cắt nhỏ nhất}).$$

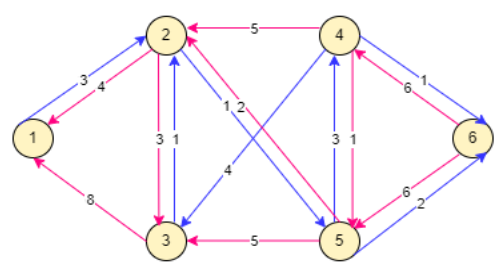
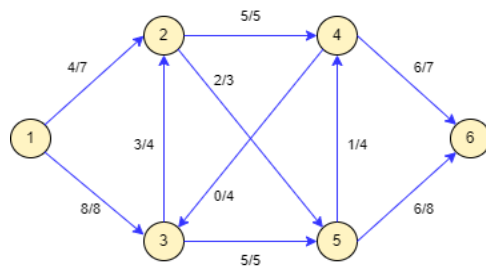
4.3. Thuật toán Ford - Fulkerson

4.3.1. Xây dựng đồ thị dư (Residual Graph)

Cho f là một luồng trong mạng $G = (V, E)$.

Ta xây dựng đồ thị dư $G_f = (V, E_f)$ với tập cung E_f và trọng số trên các cung = residual capacity = original capacity - current flow, cụ thể quy tắc như sau:

- Nếu (u, v) chưa sử dụng thông lượng, tức $f(u, v) = 0$, thì giữ nguyên nó vào E_f với trọng số = $c(u, v)$
- Nếu (u, v) còn dư thông lượng thì thêm vào E_f :
 - Cung thuận (u, v) có trọng số = $c(u, v) - f(u, v)$
 - Cung nghịch (v, u) có trọng số = $f(u, v)$
- Nếu (u, v) đã sử dụng hết thông lượng thì thêm vào E_f :
 - Cung nghịch (v, u) có trọng số = $f(u, v) = c(u, v)$



Đường tăng luồng (Augmenting path):

- Là mọi đường đi P từ source đến sink trên đồ thị dư G_f .
- Là một chuỗi các cạnh nối từ source đến sink, cho phép ta tăng thêm tải trọng luồng bằng cách đẩy thêm giá trị luồng dọc theo đường này.
- Giá trị luồng đẩy thêm = trọng số nhỏ nhất trong các trọng số của các cung trên đường P .

Định lý: Cho f là một luồng trong mạng $G = (V, E)$, các phát biểu sau đây là tương đương:

- f là luồng cực đại trong mạng
- Không tìm được đường tăng luồng P nào nữa
- $val(f) = c(X|Y)$, với $(X|Y)$ là một lát cắt nào đó của mạng

4.3.2. Thuật toán Ford - Fulkerson

Mục tiêu: Giải bài toán luồng cực đại Max-Flow tìm luồng f cho mạng $G = (V, E)$ sao cho:

- Thỏa ràng buộc thông lượng trên mỗi cung
- Bảo toàn luồng tại mỗi đỉnh trung gian
- Tối đa hóa tổng luồng từ source đến sink

Ý tưởng:

- Khởi tạo luồng bằng 0
- Lặp cho đến khi không còn đường tăng:
 - Tìm đường tăng luồng P trong đồ thị dư $G_f = (V, E_f)$ (sử dụng bất kỳ thuật toán tìm đường đi nào, chẳng hạn BFS/DFS/...)

- Xác định giá trị tăng thêm của luồng = $\min(\text{residual capacity}) = \min(c_f(u, v))$, với $\forall(u, v) \in P$
- Tăng luồng dọc theo P

Minh họa:

- <https://visualgo.net/en/maxflow>
- <https://www.geeksforgeeks.org/dsa/ford-fulkerson-algorithm-for-maximum-flow-problem/>

Pseudocode:

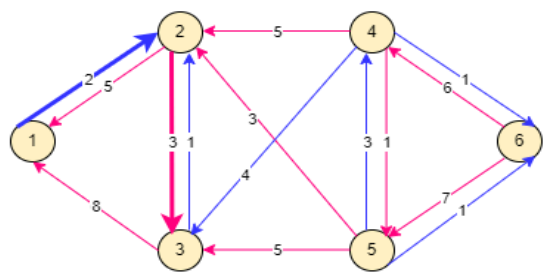
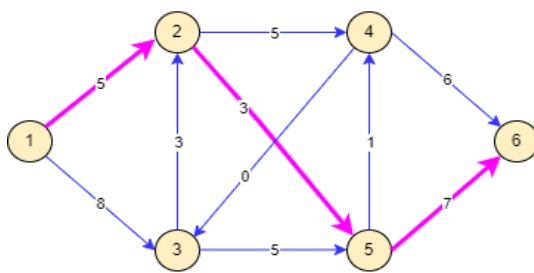
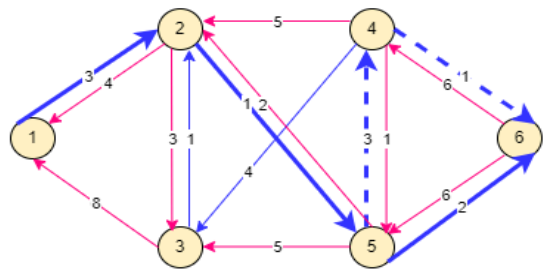
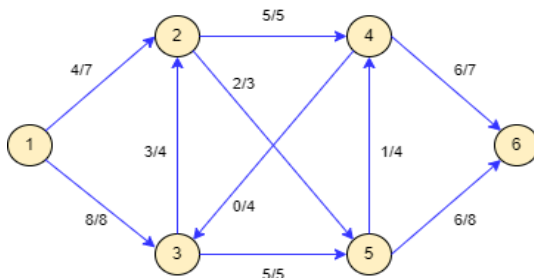
```

Ford_Fulkerson(G, source, sink):
    for each edge (u, v) in G:
        f(u, v) = 0

    while tồn tại đường tăng luồng P từ source đến sink trong
    đồ thị dư G_f:
        delta = min{c_f(u, v) trên mọi cạnh (u, v) thuộc P}
        for mỗi cạnh (u, v) trong P:
            f(u, v) = f(u, v) + delta
            f(v, u) = f(v, u) - delta

    return tổng luồng ra khỏi source

```



Trong python, sử dụng thư viện `networkx` để tìm luồng cực đại trong mạng theo thuật toán Ford-Fulkerson/ Edmonds-Karp:

```

In [30]: import networkx as nx

G = nx.DiGraph()
G.add_edge('1', '2', capacity=7)
G.add_edge('1', '3', capacity=8)
G.add_edge('3', '2', capacity=4)
G.add_edge('2', '4', capacity=5)
G.add_edge('3', '5', capacity=5)
G.add_edge('2', '5', capacity=3)
G.add_edge('4', '3', capacity=4)
G.add_edge('5', '4', capacity=4)

```

```
G.add_edge('4', '6', capacity=7)
G.add_edge('5', '6', capacity=8)
```

```
# Tính max flow:
flow_value, flow_dict = nx.maximum_flow(G, '1', '6')
print("Max flow:", flow_value)
print("Flow distribution:")
print(flow_dict)
```

Max flow: 13

Flow distribution:

```
{'1': {'2': 7, '3': 6}, '2': {'4': 5, '5': 3}, '3': {'2': 1, '5': 5}, '4': {'3': 0, '6': 5}, '5': {'4': 0, '6': 8}, '6': {}}
```

4.4. Thuật toán Edmonds - Karp

Cải tiến từ thuật toán Ford - Fulkerson trong tìm đường tăng luồng P , bằng cách sử dụng BFS.

4.5. Thuật toán Dinic

Cải tiến thuật toán Edmonds - Karp xử lý nhiều đường tăng luồng P cùng lúc thông qua level graph để tìm ra luồng chặn.

Đồ án

Yêu cầu

Quy định nộp bài

Quy định chấm bài

Tài liệu tham khảo

- Võ Nam Thực Đoan, Lê Phúc Lữ, Bài tập và Thực hành Toán học tổ hợp, bản thảo, 2025.
- Bộ bài giảng Toán học tổ hợp của PGS.TS. Đinh Ngọc Thanh - ThS. Võ Nam Thực Đoan, ThS. Lê Phúc Lữ.
- Bộ bài giảng Cấu trúc dữ liệu và giải thuật của ThS. Văn Chí Nam.
- Alan Tucker, Applied Combinatorics, 6th edition, John Wiley & Sons, Inc, 2012.
- Trần Đan Thư, Dương Anh Đức, Lý Thuyết Đồ Thị, NXB Đại Học Quốc Gia Tp. Hồ Chí Minh, 2008.
- <https://www.geeksforgeeks.org/dsa/graph-data-structure/>
- <https://www.cs.usfca.edu/~galles/visualization/>
- <https://visualgo.net/en>

--- HẾT ---